Search, Substitute, & Global Commands

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`/pattern and ?pattern`	Search forward (/) or backward (?) in current buffer; press n for next match, N for previous EXAMPLE: `/\v<[A-Z]\w+>`
`:%s/pattern/replace/g`	Substitute all occurrences of pattern across entire file (% = lines 1,$); /g flag replaces all occurrences per line EXAMPLE: `:%s/\vfoo/bar/g`
`:%s/pattern/replace/gc`	Substitute with interactive confirmation prompt for each match (y=yes, n=no, a=all, q=quit, l=last) EXAMPLE: `:%s/old_fn/new_fn/gc`
`:'<,'>s/pattern/replace/g`	Substitute only within currently visually selected lines (auto-inserted after pressing : in Visual mode) EXAMPLE: `:'<,'>s/\s\+$//`
`:g/pattern/d`	Global delete — deletes every line matching pattern across the entire buffer EXAMPLE: `:g/^\s*$/d (deletes all blank lines)`
`:v/pattern/d (or :g!/pattern/d)`	Inverted global delete — deletes every line that does NOT match pattern EXAMPLE: `:v/^ERROR/d (keeps only ERROR lines)`
`:g/pattern/norm command`	Executes Normal mode keystrokes on every line matching pattern EXAMPLE: `:g/^#/norm >> (indents all comment lines)`
`:s/pattern/replace/e`	The 'e' flag suppresses error messages if pattern is not found (useful in mappings and scripts) EXAMPLE: `:%s/\s\+$//e`

Vim Magic Modes (\v, \m, \M, \V)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`\v (Very Magic)`	Very magic — all punctuation characters (+, ?, (,), {, }, |, <, >) have special regex meaning WITHOUT backslashes EXAMPLE: `/\v(\d{3})-(\d{4}) instead of /\(\d\{3\}\)-\(\d\{4\}\)`
`\m (Magic – Default)`	Default Vim mode — only . * ^ $ [ ] are special; +, ?, (,), {, }, | require a leading backslash (\+, \?, \() EXAMPLE: `/\(\d\+\) matches 1+ digits in default magic mode`
`\M (Nomagic)`	No magic — only ^ and $ have special meaning; all other metacharacters (. * [etc.) are treated literally EXAMPLE: `/\Mfoo.bar matches literal 'foo.bar'`
`\V (Very Nomagic)`	Very no-magic — completely literal search; only the backslash (\) retains special escape meaning EXAMPLE: `/\V[a-z]*(foo) matches literal '[a-z]*(foo)'`
`\c and \C`	Overrides 'ignorecase' setting: \c forces case-insensitive search; \C forces case-sensitive search EXAMPLE: `/\v<admin>\c matches 'Admin', 'ADMIN', 'admin'`

Vim Character Classes & Multi-Line Tokens

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`\< and \>`	Word boundary delimiters — \< matches start of a word, \> matches end of a word (synonym for \b) EXAMPLE: `/\<cat\> matches 'cat' but not 'catch' or 'bobcat'`
`\k and \K`	Keyword character (defined by 'iskeyword' option, includes letters, digits, and underscores) and non-keyword EXAMPLE: `/\k\+ matches identifier`
`\i and \I`	Identifier character (valid in programming language identifiers) and non-identifier EXAMPLE: `/\i\+ matches identifier token`
`\f and \F`	File name character (valid characters in file paths per 'isfname' setting) and non-filename EXAMPLE: `/\f\+ matches path like '/usr/local/bin'`
`\s and \S`	Whitespace character (space or tab) and non-whitespace EXAMPLE: `/^\s\+ matches indented lines`
`\d and \D`	Digit [0-9] and non-digit [^0-9] EXAMPLE: `/\d\{4\} matches 4 digits`
`\a, \u, \l`	Alphabetic letter (\a), Uppercase letter (\u), and Lowercase letter (\l) EXAMPLE: `/\u\l\+ matches capitalized words`
`\_s, \_d, \_w, \_.`	Multi-line character classes including newline — _s matches whitespace OR newline; _. matches any character including newline EXAMPLE: `/function\_.\{-}end matches multi-line block`

Match Boundaries (\zs, \ze) & Lookarounds

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`\zs`	Start of match — sets start position of actual match, discarding everything matched to the left (like \K in Perl) EXAMPLE: `:%s/width: \zs\d\+/100/g (changes only the number)`
`\ze`	End of match — sets end position of actual match, matching text to the right without replacing it EXAMPLE: `:%s/\d\+\zepx/24/g (changes only the number before 'px')`
`pattern\@=`	Positive lookahead assertion — matches if pattern matches immediately to the right EXAMPLE: `/foo\(bar\)\@= (or in \v: /foo(bar)@=)`
`pattern\@!`	Negative lookahead assertion — matches if pattern does NOT match immediately to the right EXAMPLE: `/foo\(bar\)\@! matches 'foo' not followed by 'bar'`
`pattern\@<=`	Positive lookbehind assertion — matches if pattern matches immediately to the left EXAMPLE: `/\(foo\)\@<=bar matches 'bar' preceded by 'foo'`
`pattern\@<!`	Negative lookbehind assertion — matches if pattern does NOT match immediately to the left EXAMPLE: `/\(foo\)\@<!bar matches 'bar' not preceded by 'foo'`
`\%^ and \%$`	Start of buffer (\%^) and absolute end of buffer (\%$) anchors EXAMPLE: `/\%^.* matches first line of file`
`\%23l and \%>20l`	Line position matching — \%23l matches only on line 23; \%>20l matches only on lines after line 20 EXAMPLE: `/\%23l\s*error matches 'error' on line 23`
`\%V`	Visual area anchor — matches only within the area selected in the last Visual mode EXAMPLE: `:%s/\%Vfoo/bar/g replaces 'foo' only inside visual block`

Replacement Field Tokens & Expressions (\=)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`\0 or &`	Inserts the entire matched text into replacement string EXAMPLE: `:%s/\v<\w+>/[&]/g (wraps words in brackets)`
`\1, \2, \3 ...`	Inserts text captured by 1st, 2nd, 3rd sub-expression groups EXAMPLE: `:%s/\v(\w+) (\w+)/\2, \1/g`
`\r`	Inserts an actual newline / line break in replacement (NOTE: \n in replacement inserts a null byte \x00!) EXAMPLE: `:%s/, /\r/g (splits comma list into separate lines)`
`\t`	Inserts a tab character into replacement EXAMPLE: `:%s/    /\t/g (converts 4 spaces to tabs)`
`\U and \L with \E`	Converts replacement text to UPPERCASE (\U) or lowercase (\L) until \E terminator EXAMPLE: `:%s/\v(\w+)/\U\1\E/g (converts words to UPPERCASE)`
`\u and \l`	Converts only the single next character to uppercase (\u) or lowercase (\l) EXAMPLE: `:%s/\v<(\w)(\w+)/\u\1\L\2\E/g (Title Cases words)`
`\= (Expression Replacement)`	Evaluates Vim script expression — submatch(0) returns whole match, submatch(1) returns group 1 EXAMPLE: `:%s/\v\d+/\=submatch(0)*2/g (doubles all numbers in file)`
`\~`	Inserts the replacement string from the previous substitute command EXAMPLE: `:%s/foo/\~/g`